

Chapter 3: Dot Net Framework

The .NET Framework is a revolutionary platform created by Microsoft for developing applications. The .NET Framework enables the creation of **Windows applications, Web applications, Web services**, and pretty much anything else we can think of. Also, with Web applications it's worth noting that these are, by definition, multiplatform applications, since any system with a Web browser can access them. With the recent addition of Silverlight, this category also includes applications that run inside browsers on the client, as well as applications that merely render Web content in the form of HTML.

The .NET Framework has been designed so that it can be used from any language, including C# as well as C++, Visual Basic, JScript, and even older languages such as COBOL. For this to work, .NET-specific versions of these languages have also appeared, and more are being released all the time. Not only do all of these have access to the .NET Framework, but they can also communicate with each other. It is perfectly possible for C# developers to make use of code written by Visual Basic programmers, and vice versa. All of this provides an extremely high level of versatility and is part of what makes using the .NET Framework such an attractive prospect.

What's in the .NET Framework?

The .NET Framework consists primarily of a gigantic library of code that we use from our client languages (such as C#) using object-oriented programming (OOP) techniques. This library is categorized into different modules—we use portions of it depending on the results we want to achieve. For example, one module contains the building blocks for **Windows applications, another for network programming, and another for Web development**. Some modules are divided into more specific submodules, such as a module for building Web services within the module for Web development.

The intention is for different operating systems to support some or all of these modules, depending on their characteristics. A PDA, for example, would include support for all the core .NET functionality but is unlikely to require some of the more esoteric modules. Part of the .NET Framework library defines some basic types. A type is a representation of data, and specifying some of the most fundamental of these (such as “a 32-bit signed integer”) facilitates interoperability between languages using the .NET Framework. This is called the **Common Type System (CTS)**. As well as supplying this library, the .NET Framework also includes the **.NET Common Language Runtime (CLR)**, which is responsible for maintaining the execution of all applications, developed using the .NET library.

.NET can be understood as a runtime environment and a comprehensive base class library. The runtime layer is properly referred to as the **Common Language Runtime**, or CLR. The primary role of the CLR is to **locate, load, and manage .NET types** on our behalf. The CLR also takes care of a number of low-level details such as **memory management, application hosting, handling threads, and performing various security checks**.

Another building block of the .NET platform is the **Common Type System**, or CTS. The CTS Specification fully describes all possible data types and programming constructs supported by the runtime, specifies how these entities can interact with each other, and details how they are represented in the .NET metadata format.

Understand that a given .NET-aware language might not support each and every feature defined by the CTS. The **Common Language Specification**, or CLS, is a related specification that defines a

subset of common types and programming constructs that all .NET programming languages can agree on. Thus, if we build .NET types that only expose CLS-compliant features, we can rest assured that all .NET-aware languages can consume them. Conversely, if we make use of a data type or programming construct that is outside of the bounds of the CLS, we cannot guarantee that every .NET programming language can interact with our .NET code library. The following figure shows that the .Net Framework.

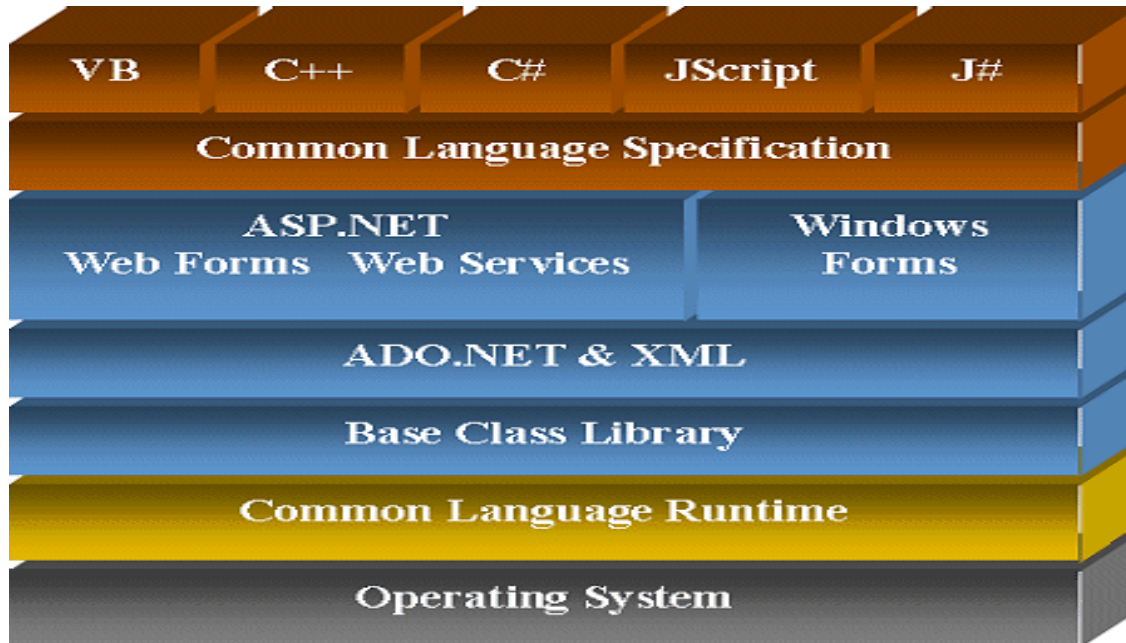


Fig: .Net Framework

The Role of the Base Class Libraries

In addition to the CLR and CTS/CLS specifications, the .NET platform provides a base class library that is available to all .NET programming languages. Not only does this base class library encapsulate various primitives such as **threads, file input/output (I/O), graphical rendering systems, and interaction with various external hardware devices**, but it also provides support for a number of services required by most real-world applications.

For example, the base class libraries define types that *facilitate database access, manipulation of XML documents, programmatic security, and the construction of web-enabled as well as traditional desktop and console-based front ends*. From a high level, we can visualize the relationship between the CLR, CTS, CLS, and the base class library, as shown in Figure.

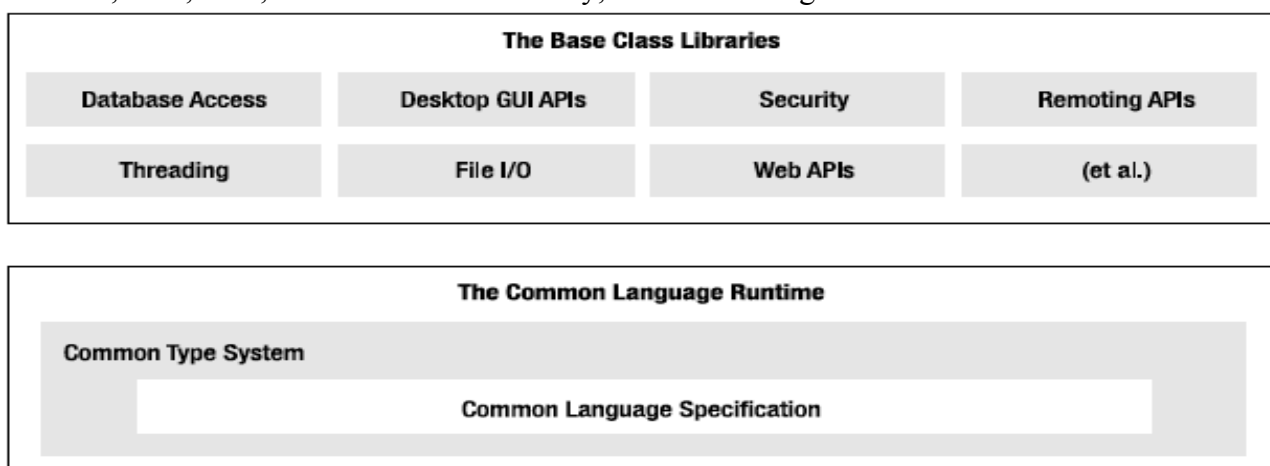


Fig: Relationship between CLS, CTS, CLR and base class library.

CIL and JIT

When we compile code that uses the .NET Framework library, we don't immediately create operating-system-specific **native code**. Instead, we compile our code into **Common Intermediate Language (CIL) code**. This code isn't specific to any operating system (OS) and isn't specific to C#. Other .NET languages — Visual Basic .NET, for example — also compile to this language as a first stage. This compilation step is carried out by VS or VCE when we develop C# applications.

Obviously, more work is necessary to execute an application. That is the job of a **just-in-time (JIT)** compiler, which compiles *CIL into native code* that is specific to the OS and machine architecture being targeted. Only at this point can the OS execute the application. The just-in-time part of the name reflects the fact that CIL code is compiled only when it is needed.

In the past, it was often necessary to compile our code into several applications, each of which targeted a specific operating system and CPU architecture. Typically, this was a form of optimization (to get code to run faster on an AMD chipset, for example), but at times it was critical (for applications to work in both Win9x and WinNT/2000 environments, for example). This is now unnecessary, because JIT compilers (as their name suggests) use CIL code, which is independent of the *machine, operating system, and CPU*. Several JIT compilers exist, each targeting a different architecture, and the appropriate one is used to create the native code required.

The beauty of all this is that it requires a lot less work on our part - in fact, we can forget about system-dependent details and concentrate on the more interesting functionality of our code.

Assemblies

When we compile an application, the CIL code created is stored in an *assembly*. *Assemblies include both executable application files that we can run directly from Windows without the need for any other programs (these have a .exe file extension) and libraries (which have a .dll extension) for use by other applications.*

In addition to containing CIL, assemblies also include meta information (that is, information about the information contained in the assembly, also known as metadata) and optional resources (additional data used by the CIL, such as sound files and pictures). The meta information enables assemblies to be fully self-descriptive. We need no other information to use an assembly, meaning we avoid situations such as failing to add required data to the system registry and so on, which was often a problem when developing with other platforms.

This means that deploying applications is often as simple as copying the files into a directory on a remote computer. Because no additional information is required on the target systems, we can just run an executable file from this directory (assuming the .NET CLR is installed).

Of course, We won't necessarily want to include everything required to run an application in one place. We might write some code that performs tasks required by multiple applications. In situations like that, it is often useful to place the reusable code in a place accessible to all applications. In the .NET Framework, this is the **global assembly cache (GAC)**. Placing code in the GAC is simple - we just place the assembly containing the code in the directory containing this cache.

Managed Code

The role of the CLR doesn't end after we have compiled our code to CIL and a JIT compiler has compiled that to native code. Code written using the .NET Framework is managed when it is

executed (a stage usually referred to as runtime). This means that the CLR looks after our applications by *managing memory, handling security, allowing cross-language debugging*, and so on. By contrast, applications that do not run under the control of the CLR are said to be *unmanaged*, and certain languages such as C++ can be used to write such applications, which, for example, access low-level functions of the operating system. However, in C# we can write only code that runs in a managed environment. We will make use of the managed features of the CLR and allow .NET itself to handle any interaction with the operating system.

Garbage Collection

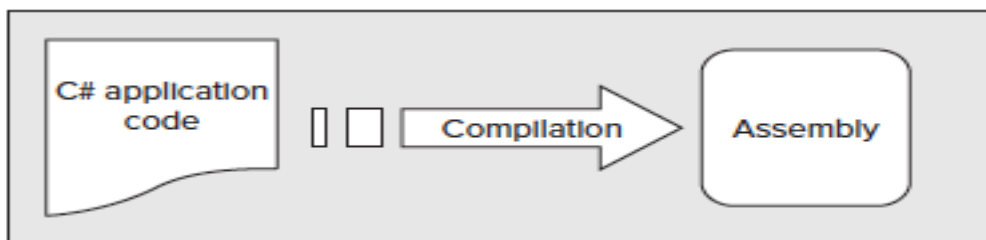
One of the most important features of managed code is the concept of garbage collection. This is the .NET method of making sure that the memory used by an application is freed up completely when the application is no longer in use. Prior to .NET this was mostly the responsibility of programmers, and a few simple errors in code could result in large blocks of memory mysteriously disappearing as a result of being allocated to the wrong place in memory. That usually meant a progressive slowdown of our computer followed by a system crash.

.NET garbage collection works by periodically inspecting the memory of our computer and removing anything from it that is no longer needed. There is no set time frame for this; it might happen thousands of times a second, once every few seconds, or whenever, but we can rest assured that it will happen.

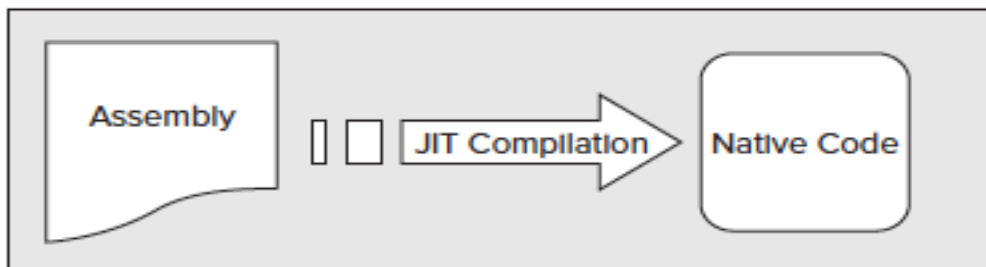
There are some implications for programmers here. Because this work is done for us at an unpredictable time, applications have to be designed with this in mind. Code that requires a lot of memory to run should tidy itself up, rather than wait for garbage collection to happen, but that isn't as tricky as it sounds.

Summary of steps required to create a .NET application

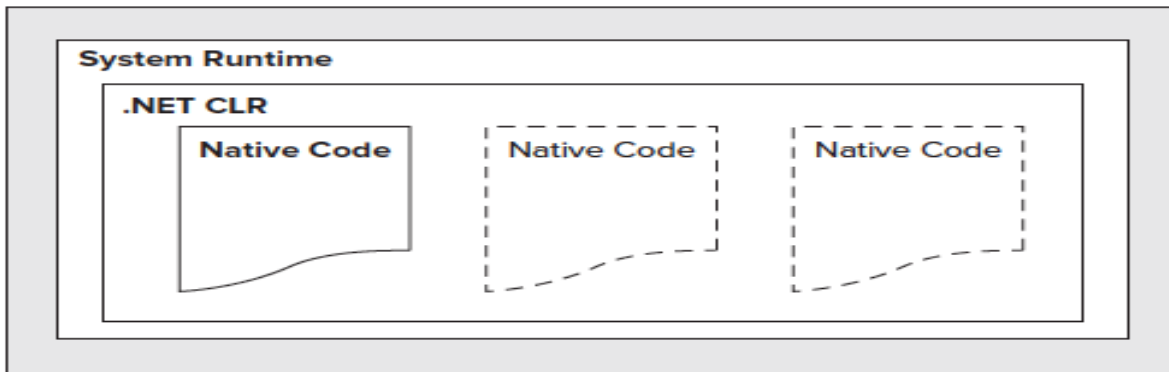
1. Application code is written using a .NET-compatible language such as C#.
2. That code is compiled into CIL, which is stored in an assembly.



3. When this code is executed (either in its own right if it is an executable or when it is used from other code), it must first be compiled into native code using a JIT compiler.



4. The native code is executed in the context of the managed CLR, along with any other running applications or processes.



Linking

The C# code that compiles into CIL in step 2 needn't be contained in a single file. It's possible to split application code across multiple source code files, which are then compiled together into a single assembly. This extremely useful process is known as **linking**. It is required because it is far easier to work with several smaller files than one enormous one. We can separate out logically related code into an individual file so that it can be worked on independently and then practically forgotten about when completed. This also makes it easy to locate specific pieces of code when we need them and enables teams of developers to divide the programming burden into manageable chunks, whereby individuals can "check out" pieces of code to work on without risking damage to otherwise satisfactory sections or sections other people are working on.

Windows Forms

Since the release of the .NET platform (circa 2001), the base class libraries have included a particular API named Windows Forms, represented primarily by the **System.Windows.Forms.dll** assembly. The Windows Forms toolkit provides the types necessary to build desktop graphical user interfaces (GUIs), create custom controls, manage resources (e.g., string tables and icons), and perform other desktop centric programming tasks. In addition, a separate API named GDI+ (represented by the **System.Drawing.dll** assembly) provides additional types that allow programmers to generate 2D graphics, interact with networked printers, and manipulate image data. The Windows Forms (and GDI+) APIs remain alive and well within the .NET 4.0 platform, and they will exist within the base class library for quite some time (arguably forever). However, Microsoft has shipped a brand new GUI toolkit called Windows Presentation Foundation (WPF) since the release of .NET 3.0. WPF provides a massive amount of horsepower that we can use to build bleeding-edge user interfaces, and it has become the preferred desktop API for today's .NET graphical user interfaces. When we need to create more traditional business UIs that do not require an assortment of bells and whistles, the Windows Forms API can often fit the bill.

The Windows Forms Namespaces

The Windows Forms API consists of hundreds of types (e.g., classes, interfaces, structures, enums, and delegates), most of which are organized within various namespaces of the **System.Windows.Forms.dll** assembly.

Far and away the most important Windows Forms namespace is *System.Windows.Forms*. At a high level, we can group the types within this namespace into the following broad categories:

- Core infrastructure: These are types that represent the core operations of a Windows Forms program (e.g., Form and Application) and various types to facilitate interoperability with legacy ActiveX controls, as well as interoperability with new WPF custom controls.
- Controls: These are types used to create graphical UIs (e.g., Button, MenuStrip, ProgressBar, and DataGridView), all of which derive from the Control base class. Controls

are configurable at design time and are visible (by default) at runtime.

- Components: These are types that do not derive from the Control base class, but still may provide visual features to a Windows Forms program (e.g., ToolTip and ErrorProvider). Many components (e.g., the Timer and System.ComponentModel.BackgroundWorker) are not visible at runtime, but can be configured visually at design time.
- Common dialog boxes: Windows Forms provides several canned dialog boxes for common operations (e.g., OpenFileDialog, PrintDialog, and ColorDialog).

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
// The minimum required windows forms namespaces.
using System.Windows.Forms;
namespace SimpleWinFormsApp
{
    // This is the application object.
    class Program
    {
        static void Main(string[] args)
        {
            Application.Run(new MainWindow());
        }
    }
    // This is the main window.
    class MainWindow : Form {}
}
```

This code represents the absolute simplest Windows Forms application and will produce following output.



Windows Presentation Foundation (WPF)

WPF, or Windows Presentation Foundation, is a graphical system for rendering user interfaces. It provides great flexibility in how we can lay out and interact with our applications. With Common Language Runtime (CLR) at its core, we can use C# or any other CLR language to communicate with user interface elements and develop application logic. The advantages of WPF for our application are its *rich data binding and visualization support and its design flexibility and styling*. WPF enables us to create an application that is more usable to our audience. It gives us the power to design an application that would previously take extremely long development cycles and a calculus genius to implement.

WPF's graphics capabilities make it the perfect choice for data visualization. Take, for instance, the standard drop-down list (or combo box). Its current use is to enable the user to choose a single item from a list of items. For this example, suppose we want the user to select a car model for purchase.

The standard way of displaying this choice is to display a drop-down list of car model names from which users can choose. There is a fundamental usability problem with this common solution: Users are given only a single piece of information from which to base their decision — the text that is used to represent the item in the list.

For the power user (or car fanatic) this may not be an issue, but other users need more than just a model name to make an educated decision on the car they wish to purchase. This is where WPF and its data visualization capabilities come into play.

A template can be provided to define how each item in the drop-down list is rendered. The template can contain any visual element, such as images, labels, text boxes, tooltips, drop shadows, and more.

XAML

WPF enables us to build user interfaces declaratively. XAML forms the foundation of WPF. XAML is similar to HTML in the sense that interface elements are defined using a tag-based syntax. XAML is XML-based and as such it must be **well formed**, meaning all opening tags require closing tags, and all elements and attributes contained in the document must validate strictly against the specified **schemas**. By default, when creating a WPF application in Visual Studio 2010, the following schemas are represented in generated XAML files:

- <http://schemas.microsoft.com/winfx/2006/xaml/presentation>: This schema represents the default Windows Presentation Framework namespace.
- <http://schemas.microsoft.com/winfx/2006/xaml>: This schema represents a set of classes that map to CLR objects. Most CLR objects can be expressed as XAML elements. XAML elements are mapped to classes; attributes are mapped to properties or events.

At runtime when a XAML element is processed, the default constructor for its underlying class is called, and the object is instantiated; its properties and events are set based on the attribute values specified in XAML.

When creating a WPF application from Visual Studio 2010, an Application template creates two XAML files along with their respective code-behind files: *App.xaml* (*App.xaml.cs*) and *MainWindow.xaml* (*MainWindow.xaml.cs*).

App.xaml represents the entry-point of the application. This is where application-wide (globally scoped) resources and the startup window are defined. *Resources* are a keyed collection of reusable objects. Resources can be created and retrieved using both XAML and C#. Resources can be anything — **data templates, arrays of strings, or brushes used to color the background of text boxes**. Resources are also scoped, meaning they can be available to the entire Application (global), to the Window, to the User Control, or even to only a specific control.

```
<Application x:Class="MyFirstWPFApplication.App"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    StartupUri="MainWindow.xaml">
    <Application.Resources>
    </Application.Resources>
</Application>
```

The above code is the XAML that was generated by the WPF Application template in Visual Studio. Note that the WPF namespaces are defined. The namespace that represents the CLR objects will be distinguished in the XAML file with the **.x** prefix.

The **StartupUri** value defines the window that will be displayed after the application is executed. In this case, the **MainWindow.xaml** window will be displayed. The **x:Class** attribute defines the C# code-behind file of this XAML file. If we open **App.xaml.cs**, we will see that its class name is **App** and it inherits from the **Application** class (which is the root element of the XAML file).

WPF brings not only a dramatic shift to the look and feel of Windows applications but also changes the manner of development. The days of dragging and dropping controls from the toolbox onto a form are long gone. Even though it is still possible to drag and drop in WPF, we will find yourself better off and much happier if we work in XAML directly.

What was once difficult is now relatively simple. For example, in traditional Windows applications, when the user changes the size of the form, the controls typically stay huddled in their corner and a large area of empty canvas is displayed. The only cure for this was a lot of custom code or expensive third-party controls. WPF brings the concept of flow layout from the Web into the Windows world.

- A Form in WinForms is referred to as a Window in WPF.
- Anything placed on a WinForms Form is called a control, whereas items placed on a WPF Window are referred to as UIElements.
- Panels are WPF UIElements used for layout.
- A Control in WPF is a UIElement that can receive focus and respond to user events.
- A Content control in WPF can contain only a single item, which can in turn be other UIElements.
- The WPF Window class is a specialized Content control.

Instead of depending on screen resolution, WPF measures UI Elements in Device Independent Units (DIUs) that are based on the system DPI. This enables a consistent look between many different hardware configurations.

WPF layout is based on relative values and is adjusted at runtime. When we place controls in a layout container, the rendering engine considers the height and width only as “suggested” values. Location is defined in relation to other controls or the container. Actual rendering is a two-step process that starts with measuring all controls (and querying them for their preferred dimensions) and then arranging them accordingly. If controls could speak, the conversation might go something like this: Layout Engine: “Control, how much space would you like to have?”

<This is the Measure Stage>

Control: “I would like 50 DIUs for height, 100 DIUs for width, and a margin of 3 DIUs in the containing Grid cell.”

Layout Engine asks all other controls and layout containers.

Layout Engine: “Sorry, you can have only 40 DIUs for height, but I can grant the rest of your requests.”

<This is the Arrange Stage>

Arranging elements with layout panel

Designing a Window begins with a Layout control, or Panel. Panels are different than Content controls in that they can hold multiple items, and depending on the Panel, a significant amount of plumbing is taken care of for us. Panels control how *UIElements* relate to each other and to their containing UIElement and do not dictate absolute positioning. Most application Windows requires some combination of Panels to achieve the required user interface, so it's important to understand them all. WPF ships with six core Panels:

♦ **Stack Panel:** Stack Panels place UIElements in -wait for it- stacks. Items are placed in either a

vertical pile (the default), like a stack of DVDs, or a horizontal arrangement, like books on a shelf. It is important to understand that the order items appear in the XAML is the order they appear in the Panel — the first UIElement in the XAML appears at the top (vertical) or on the far left (horizontal)

- ✦ **Wrap Panel:** The Wrap Panel automatically wraps overflow content onto the next line(s). This is different than how a typical toolbar works, where overflow items are hidden when there isn't enough real estate to show them.
- ✦ **Dock Panel:** The Dock Panel uses attached properties to “dock” child UIElements. An important thing to remember is that child elements are docked in XAML order, which means if we have two items assigned to the left side, the first UIElement as it appears in the XAML gets the far left wall of the Panel, followed by the next item.
- ✦ **Canvas:** The Canvas is a bit of an anomaly in WPF, since it doesn't use flow layout, but goes back to fix position layout rendering.
- ✦ **Uniform Grid:** The Uniform Grid divides the layout area into equally sized cells. The number of Rows and Columns are defined in the UniformGrid XAML tag.
- ✦ **Grid:** The Grid is in fact the default panel in a Window when we add a new WPF Window to our project. The Grid divides the layout area with rows (RowDefinitions) and columns (ColumnDefinitions). The difference between the Grid and the Uniform Grid is that the Grid allows for sizing of the cells by defining RowDefinition Height and ColumnDefinition Width.

Windows Communication Foundation

Windows Communication Foundation is just that the foundation for communication between Windows computers. It just so happens that thanks to open standards like SOAP and ReST, WCF can communicate with other software systems.

Although ASMX was really designed to make public services such as, for instance, adding an API to a simple Web application, WCF is a complete distributed computing platform for Windows.

In the early days of .NET, there was a technology called .NET Remoting that replaced DCOM. DCOM was Distributed COM, or the common accepted way to communicate between distributed components. Remoting replaced it when .NET came out. Remoting basically took the principles of DCOM and migrated them to .NET.

WCF isn't like that. WCF is a complete *rethinking of distributed computing*, based on the understanding that computing is becoming more and more distributed. New protocols for communication come out every day.

The goal here then is to define the differences between ASMX and WCF, and see that WCF is a true communications protocol and that ASMX is solely for adding services to Web sites. We can use either technology for both tasks, but one is certainly more suited than the other for each.

First there was DCOM. Then there was .NET Remoting. The path to a distributed computing platform for Microsoft has been a long one. Distributed computing is a hard problem to solve; nothing bad on the Microsoft developers for continuing to hone their platform. Anyway, there is SOAP, there are Microsoft binary formats, people are creating custom HTTP

contexts: the distributed computing platform is a mess. Something needed to happen to enable us to all take our existing code and make it available across the enterprise.

The designers of WCF (largely Doug Purdy, Don Box, and crew) had two diverse issues. On one hand was ASMX, providing SOAP Web service access to business logic. On the other hand was .NET Remoting, providing Microsoft a custom, black box format for the transmission of information between components via known network protocols.

Taking a deeper look at Remoting makes the case for a comprehensive platform even more clear.

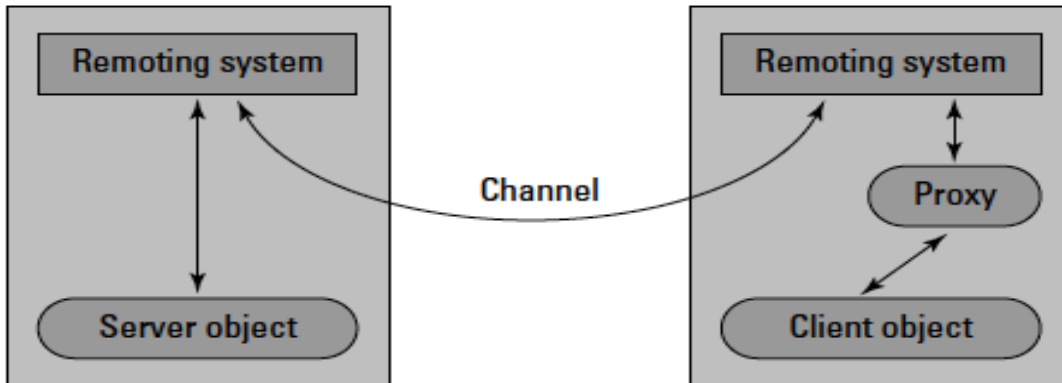


Fig: The Remoting System

If each of two systems, say a Client System and a Server System, had a similarly configured Remoting system, then they could communicate. The problem, of course, is that it was never, ever configured correctly (or so it seemed). One side of the equation or the other will make some change, and the whole system will come to a crashing halt. Clearly, there has to be some product that brings the various formats of remote access together under one umbrella. Something that would accept one block of logic and provide multiple service types.

Eventually, WCF was that solution. Starting as an add-on to .NET 2.0, WCF effectively enabled developers to make specific endpoints for a generic connector.

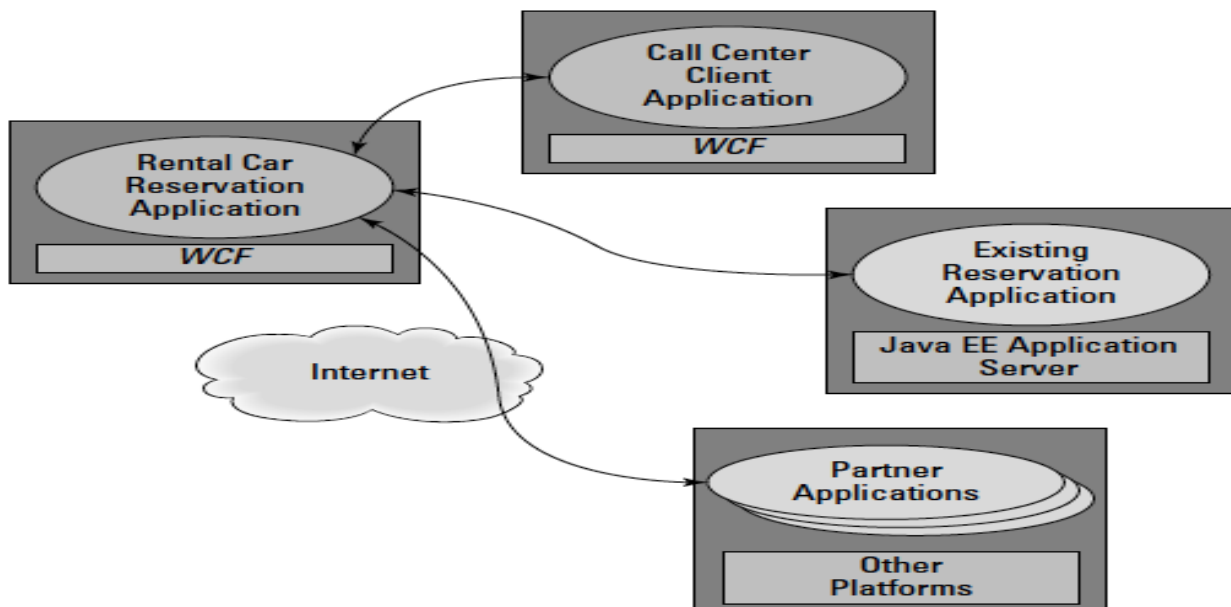


Fig: Original Problem to be solved

A car dealership is trying to build a new reservation application. The business logic needs to be both accessible to outside applications and provide a quality binary transport format for the internal communication. WCF is the answer. It uses configuration to provide various endpoints to consuming applications, from SOAP to ReST to binary associations that resemble DCOM. It

doesn't require configuration on both ends of the pipe, only on the server side. If the server serves it, the client can consume it.

Using different endpoints

The whole point of WCF is that we can write one service and then have IIS accept a lot of different protocols calling on the same code. Sure, we can do SOAP, like we did in ASMX. We also can do a lot of other protocols. Here is the breakdown of all the different endpoints, according to MSDN:

1. **BasicHttpBinding:** A binding that is suitable for communicating with WS-Basic Profile conformant Web services (for example, ASP.NET Web services [ASMX]- based services). This binding uses HTTP as the transport and text/XML as the default message encoding.
2. **WSHttpBinding:** A secure and interoperable binding that is suitable for non-duplex service contracts.
3. **WSDualHttpBinding:** A secure and interoperable binding that is suitable for duplex service contracts or communication through SOAP intermediaries.
4. **WSFederationHttpBinding:** A secure and interoperable binding that supports the WS-Federation protocol that enables organizations that is in a federation to efficiently authenticate and authorize users.
5. **NetTcpBinding:** A secure and optimized binding suitable for cross-machine communication
6. between WCF applications.
7. **NetNamedPipeBinding:** A secure, reliable, optimized binding that is suitable for on-machine communication between WCF applications.
8. **NetMsmqBinding:** A queued binding that is suitable for cross-machine communication between WCF applications.
9. **NetPeerTcpBinding:** A binding that enables secure, multiple machine communication.
10. **MsmqIntegrationBinding:** A binding that is suitable for cross-machine communication between a WCF application and existing Message Queuing applications.
11. **BasicHttpContextBinding:** A binding that is suitable for communicating with WS-Basic Profile conformant Web services that enables HTTP cookies to be used to exchange context.
12. **NetTcpContextBinding:** A secure and optimized binding suitable for cross-machine communication between WCF applications that enables SOAP headers to be used to exchange context.
13. **WebHttpBinding:** A binding used to configure endpoints for WCF Web services that are exposed through HTTP requests instead of SOAP messages.
14. **WSHttpContextBinding:** A secure and interoperable binding that is suitable for non-duplex service contracts that enables SOAP headers to be used to exchange context.

So when do you use what? Well, if you want to have two .NET applications communicate, or the layers of a single application communicate, then use NetTcpBinding. If you are cross-platform communicating, then use WSHttpBinding. Only if for some reason those don't work do you start to look into the others. Oh, and remember that you can have many endpoints for the same service. Don't use SOAP for a data layer. Just make a separate SOAP header if you need cross-platform support.